INTEGRAL DRAGON

Spec-Driven Development

The spec is the source of truth, not the prompt

Session 05 | Module 2: Spec-Driven Development & Context Control

Confidential. Prepared exclusively for Ensemble Health Partners.

THE PROBLEM

Most AI misses are specification misses — the model fails where intent is missing, not where syntax is hard

Review pain starts upstream

Vague tickets create vague plans, vague diffs, and vague tests. By the time a reviewer sees the PR, the ambiguity has multiplied through every layer. The fix isn't better prompting — it's better requirements.

Your team already uses AI often

The bottleneck is not basic access. It's reviewability, trust, governance, and consistent outputs on real work. The next maturity jump is better requirement shaping — not more model access.

Spec-Driven Development is the answer

SDD turns "please build this" into "here is what must be true when we are done." It makes AI output governable, reviewable, and predictable.

Where AI actually fails

It's not a syntax problem

The model can generate syntax, scaffolding, and candidate implementations. It cannot reliably infer unstated business rules, non-goals, operational constraints, or compliance boundaries for enterprise work.

The core insight

A structured specification narrows the solution space, lowering rework, reducing prompt thrash, and making downstream planning more stable. The spec is upstream quality control.

FRAMING

Vibe coding optimizes for motion. SDD optimizes for convergence.

Vibe Coding

Prompt → inspect → retry → patch → hope

Fast initial output. Unpredictable quality. Rework multiplies with every iteration. No reviewable contract. Fine for prototypes and weekend experiments.

Spec-Driven Development

Define target behavior, constraints, and checks → then generate code

The win is not formality for its own sake. The win is fewer loops, more predictable output, and a reviewable contract that the whole team can hold the work against.

Key distinction: Vibe coding asks "did this work?" SDD asks **"does this meet the specification?"** One is subjective. The other is auditable.

SPEC STRUCTURE

A useful engineering spec is compact and executable — not a heavyweight document

1. **Goal**
   What outcome must exist when the work is done. One sentence, observable.
2. **Constraints**
   What the agent must not violate. Security boundaries, schema stability, existing contracts.
3. **Interfaces**
   APIs, files, schemas, commands, or contracts touched by this change.
4. **Acceptance Criteria**
   Binary checks that let a reviewer say pass or fail. The control surface.
5. **Non-Goals**
   What is explicitly out of scope. Prevents the agent from widening the work.

This is not a PRD

Compact and versionable

Students need a structure covering goal, constraints, interfaces, edge cases, acceptance checks, and non-goals — not heavyweight template theater. If the spec takes longer to write than the implementation, the format is wrong.

Acceptance criteria are the control surface

If the team cannot tell whether the output passed or failed, the spec is not ready. "Feels right" is not a criterion. "Returns 401 for expired token and does not mutate existing sessions" is.

ACCEPTANCE CRITERIA

Good acceptance criteria are observable — a reviewer can verify them without asking the author

WEAK		BETTER
"Make auth more robust"	→	"Expired bearer token returns 401 with the existing error envelope"
"Improve the database layer"	→	"No database schema changes. Existing integration tests continue to pass."
"Should probably use a service class"	→	"Must use existing audit logger" — a real constraint, not design advice

The test: Write what must be true before guessing how it should be built. Lead with behavior, boundaries, and risk. Add implementation constraints only when they are **real requirements**, not preferences.

TEMPLATE

Minimum viable spec — the fields that matter, nothing more

```
## Spec: [Feature Name]

### Problem Statement
What is broken or missing

### Current Boundary
Affected area, files, services

### Inputs & Outputs
What goes in, what comes out

### Constraints & Invariants
What must not be violated

### Edge Cases & Failure Modes
What can go wrong

### Acceptance Checks
Binary pass/fail criteria

### Non-Goals
What is explicitly out of scope
```

Why this structure works

Each field answers a specific question a reviewer or agent would ask. Problem statement sets context. Constraints prevent scope creep. Acceptance checks make "done" binary. Non-goals prevent the agent from widening scope silently.

Repo reconnaissance first

Read the current code before freezing the spec. Ask Claude Code to map the auth flow, touched files, and existing conventions. Then write the spec against the system that exists, not the system you imagine.

Small specs, small PRs

A clear spec can usually be sliced into 2–3 reviewable work packets: boundary mapping, implementation change, tests and narration. Decomposition quality improves when scope is decided before implementation.

WORKFLOW

A short SDD loop beats a giant requirements ceremony

1. **Ticket or Idea**
   Start with the raw request from the backlog
2. **Read the Boundary**
   Map affected files, services, and conventions
3. **Draft the Spec**
   Goal, constraints, interfaces, acceptance checks
4. **Critique**
   Challenge the spec for ambiguity and assumptions
5. **Freeze Checks**
   Lock the acceptance criteria
6. **Hand to Planning**
   Spec feeds Plan Mode in Session 06

Tickets must be upgraded

Before code generation starts. Ambiguity should be surfaced as questions, not silently guessed away by the model.

Acceptance criteria drive tests

The checks in the spec become the test plan and review comments. No separate "what should we test?" conversation.

Small PRs become natural

When the scope is decided before implementation, decomposition into reviewable slices is straightforward.

LIVE DEMO

Turn a weak ticket into an executable spec — live in Claude Code

Demo prompt flow

```
$ claude

# Step 1: Surface ambiguity
> "Critique this ticket for ambiguity and
  hidden assumptions."

# Step 2: Rewrite as spec
> "Rewrite it as a compact markdown spec
  with binary acceptance criteria."

# Step 3: Find remaining gaps
> "List open questions that still block
  safe planning."
```

What to listen for

🔍 Did Claude surface missing business rules, or did it quietly invent them? Silent invention is the danger sign.

✅ Are the acceptance checks observable by a reviewer? Could someone verify them without asking the original author?

🔪 Did the rewrite reduce ambiguity without becoming a giant PRD? Compact is the goal.

🚀 Could this spec now feed Plan Mode safely? That's the bridge to Session 06.

Starting ticket: `"Build me a password reset endpoint."` — watch how much is underspecified.

FAILURE MODES

Common ways spec discipline breaks down — and how to catch them

Treating a user story as a spec
User stories capture intent from a stakeholder perspective. They are not engineering specifications. "As a user, I want to reset my password" has no constraints, no interfaces, and no acceptance criteria.

Criteria that sound good but can't be tested
"The endpoint should be secure" passes no test. "All inputs are validated against the OpenAPI schema and invalid payloads return 422" is testable.

Smuggling implementation into acceptance
"Use a service class pattern" is design advice, not a requirement. Acceptance criteria define what must be true, not how to build it. Keep them separate.

Forgetting non-goals
Without explicit non-goals, the agent will widen scope. "Do not modify the existing migration files" and "Do not add new dependencies" must be stated, not assumed.

Self-check: Before handing a spec to planning, ask: "Could a reviewer tell whether this work passed **without asking the original author?**"

KEY TAKEAWAY

A prompt can start work. A spec can govern work. In enterprise delivery, governing work is the job.

The agent is not the source of truth. The spec is.

If the team wants reliable AI-assisted delivery, intent must be made explicit before planning and execution begin.

Bridge to Session 06

A good spec is necessary. It is not sufficient.

Next: how Plan Mode turns a spec into a reviewable execution proposal before the repo is touched.